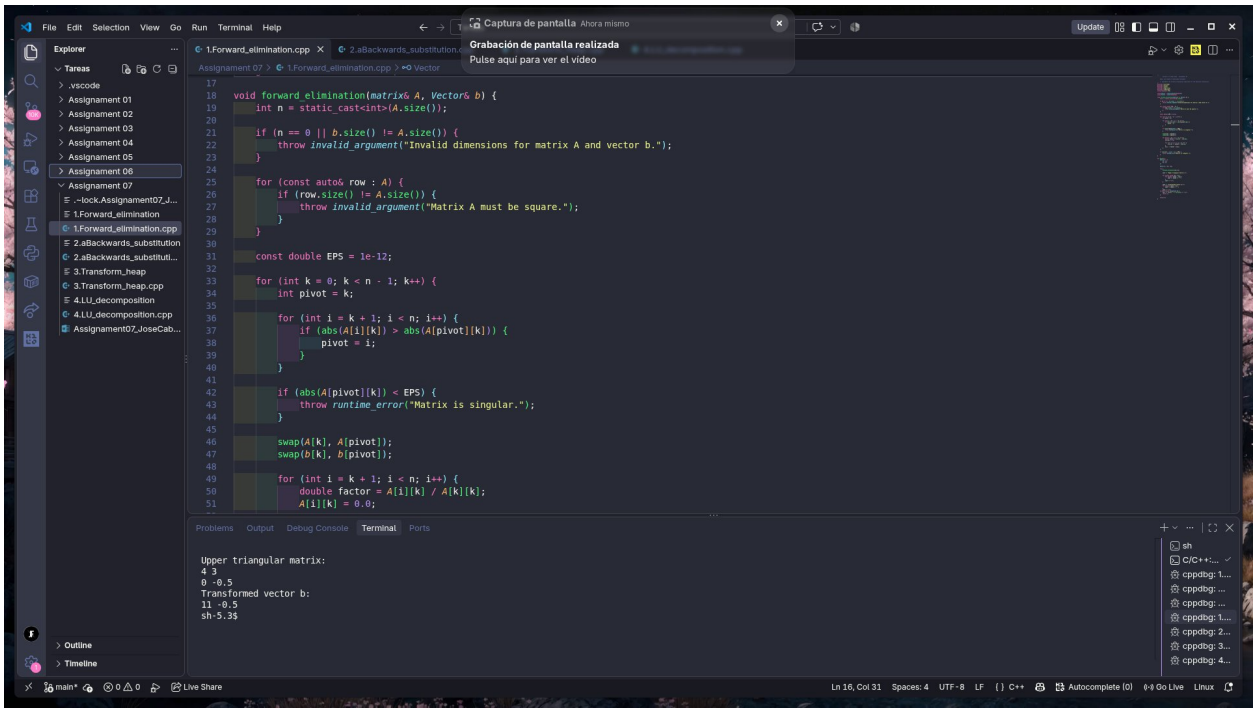
**Analysis of Algoritms**

**Assignament 07**

**Name:** Jose Ignacio Cabascango Fernandez

**Link GitHub:** https://github.com/CodeWithJose1/Analysis-of-Algorithms/tree/main/Assignament-07/src

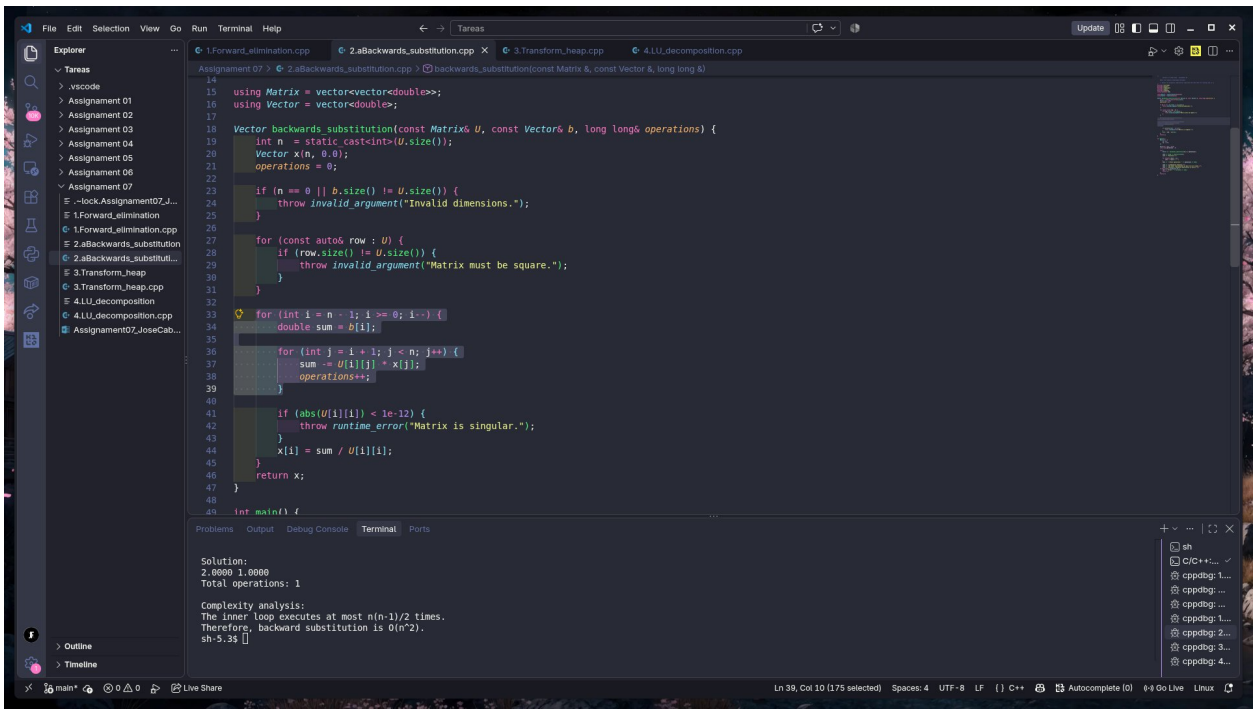
## 1. Implement the forward elimination algorithm for the Gaussian elimination.




Gaussian elimination transforms a system of linear equations into an upper triangular system using row operations. The algorithm eliminates the coefficients below the main diagonal, making the system easier to solve using backward substitution. Partial pivoting can be used to improve numerical stability.

**Time Complexity:** O(n^3), because the algorithm performs nested operations over the rows and columns of the matrix.

## 2. Design the backwards substitution algorithm and show that its running time is in O(n^2).




Backward substitution solves an upper triangular system of linear equations. It starts with the last equation and calculates the unknown variables in reverse order until all variables have been determined.

```
for (int i = n - 1; i >= 0; i--) {
        double sum = b[i];

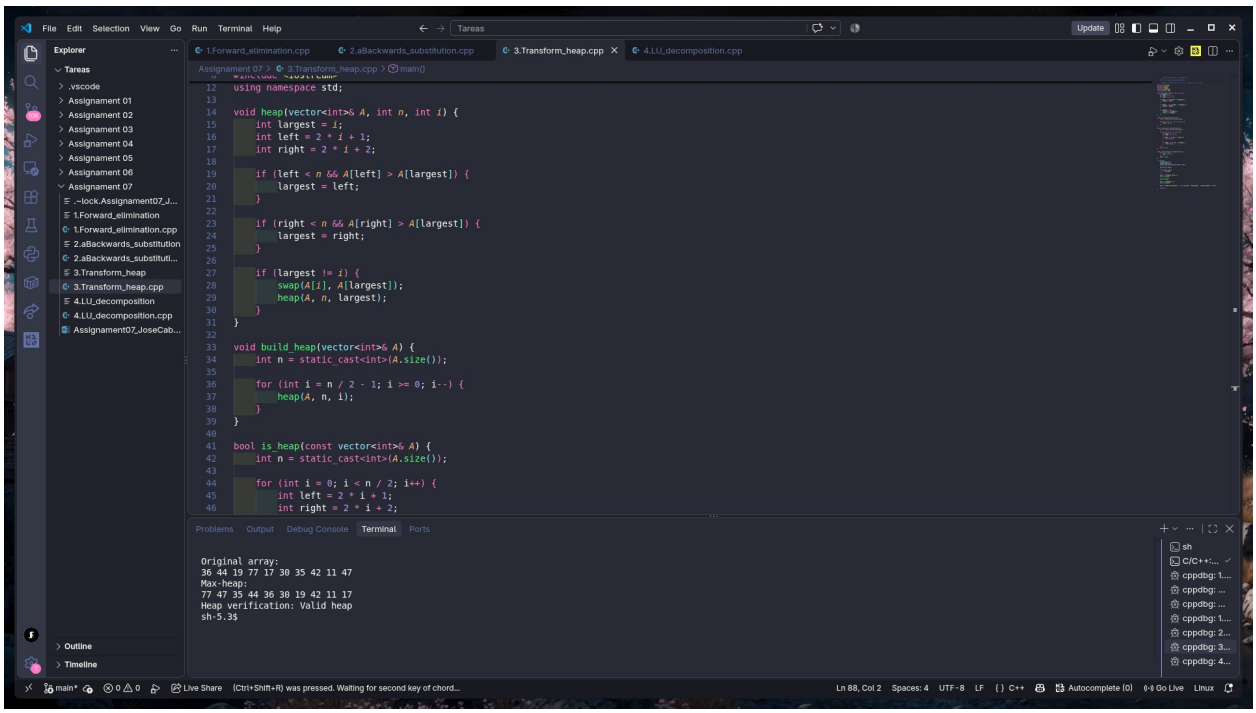
        for (int j = i + 1; j < n; j++) {
            sum -= U[i][j] * x[j];
            operations++;
        }
```

The outer loop executes (n) times, while the total number of inner-loop iterations is

$$\sum_{i=0}^{n-1} (n - 1 - i) = \frac{n(n-1)}{2}$$

Therefore, the dominant term is n^2, which corresponds to O(n^2). More precisely, its time complexity is Θ(n^2) for a dense upper triangular system.

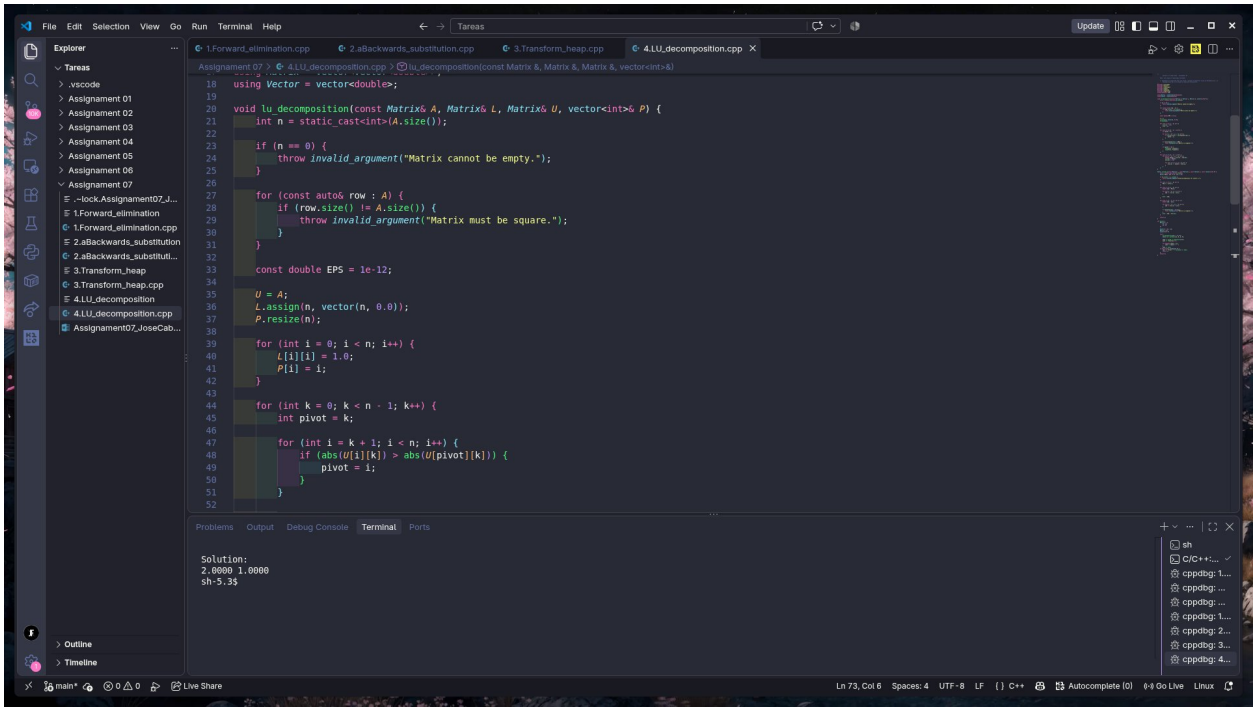
**3. Implement an algorithm to transform a random array into a heap.**




A max-heap is a complete binary tree in which each parent node is greater than or equal to its children. The implementation uses a bottom-up heap construction algorithm, applying the heap operation to internal nodes starting from the last parent and moving toward the root.

The resulting structure satisfies the max-heap property. The bottom-up construction algorithm has a time complexity of O(n), making it more efficient than inserting all elements individually into an initially empty heap, which takes O(n log n) in the worst case.

**4. Implement an algorithm that can solve a system of equations using LU decomposition. LU decomposition has to be based on Gaussian Elimination.**



LU decomposition expresses a matrix as the product of a lower triangular matrix L and an upper triangular matrix U. When partial pivoting is required, the decomposition is expressed as PA=LU, where P is a permutation matrix.

After computing the decomposition, the system is solved in two stages: forward substitution solves Ly=Pb, and backward substitution solves Ux=y.

**Time Complexity:** O(n^3) for the decomposition, while each substitution step takes O(n^2). Thus, the decomposition dominates the overall running time.